Writing recipes - going further



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



The per-recipe sysroot



Sysroot

- The *sysroot* is the logical root directory for headers and libraries
- Where *gcc* looks for headers, and *ld* looks for libraries
- Contains:
  - The kernel headers
  - The C library and headers
  - Other libraries and their headers



Optional dependencies

- Most software have a configure script that checks for libraries:
  - Fails if mandatory dependencies are not found
  - Fails if explicitly requested optional dependencies are not found
  - Enables more features if automatic dependencies are found
- Problem with automatic dependencies: result can depend on build order. Example:
  - `libpcap` built *before* `bluez5`

```
libpcap$ ./configure
...
checking for bluetooth/bluetooth.h... no
configure: Bluetooth sniffing is not supported; install bluez-lib devel to enable it
```

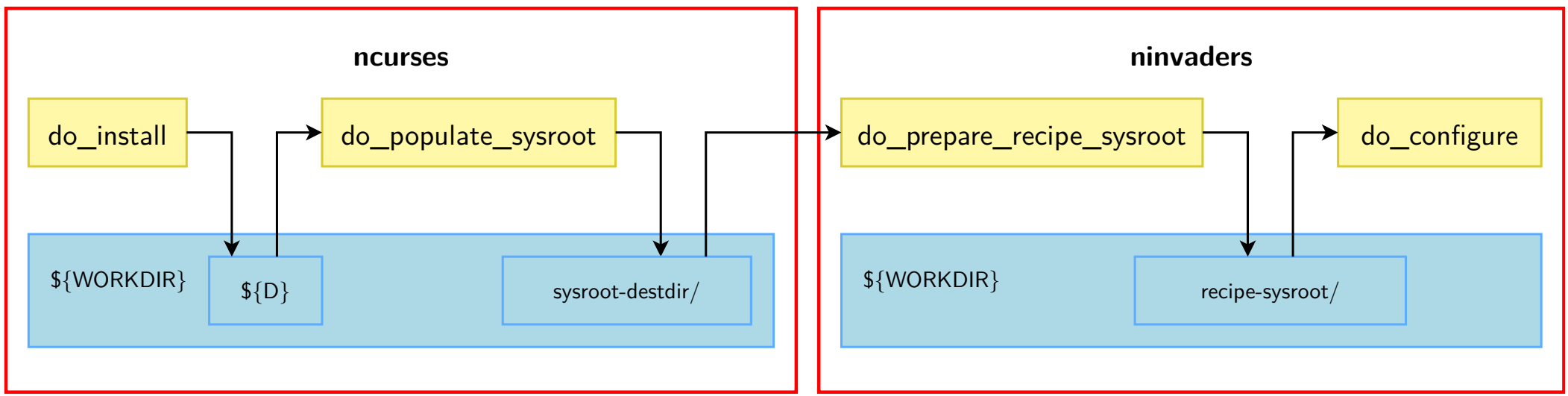
  - `libpcap` built *after* `bluez5`

```
libpcap$ ./configure
...
checking for bluetooth/bluetooth.h... yes
configure: Bluetooth sniffing is supported
```



Per-recipe sysroot

- Instead of a global sysroot, `bitbake` implements a *per-recipe sysroot*
- Main goal: reproducible build even with automatic dependencies





Per-recipe sysroot

- Before the actual build, each recipe prepares its own sysroot
  - Contains libraries and headers *only* for the recipes it DEPENDS on
  - Ensures the configuration stage will not detect libraries not explicitly listed in DEPENDS but already built for other reasons
  - `${WORKDIR}/recipe-sysroot` for target recipes
  - `${WORKDIR}/recipe-sysroot-native` for native recipes
- At the end of the build, each recipe produces its destination sysroot
  - Its own slice of sysroot, with the libraries and headers it directly provides
  - Used as input for other recipes to generate their recipe-sysroot
  - `${WORKDIR}/sysroot-destdir`



The complete sysroot

- A complete sysroot is available:
  - For each image
    - In `${WORKDIR}/recipe-sysroot` just like any recipe
  - In the SDK
    - Covered later



Using Python code in metadata



Tasks in Python

- Tasks can be written in Python when using the `python` keyword.
- Two modules are automatically imported:
  - `bb`: to access `bitbake`'s internal functions.
  - `os`: Python's operating system interfaces.
- You can import other modules using the `import` keyword.
- Anonymous Python functions are executed during parsing.
- Short Python code snippets can be written inline with the `${@<python code>}` syntax.



Accessing the datastore with Python

- The `d` variable is accessible within Python tasks.
- `d` represents the `bitbake` datastore (where variables are stored).

`d.getVar("X", expand=False)` Returns the value of `X`.
`d.setVar("X", "value")` Set `X`.
`d.appendVar("X", "value")` Append `value` to `X`.
`d.prependVar("X", "value")` Prepend `value` to `X`.
`d.expand(expression)` Expand variables in `expression`.



Python code examples

```
# Anonymous function (automatically called at parsing time)
python __anonymous() {
    if d.getVar("FOO", True) == "example":
        d.setVar("BAR", "Hello, World.")
}
# Task
python do_settime() {
    import time
    d.setVar("TIME", time.strftime('%Y%m%d', time.gmtime()))
}
# Inline Python code
do_install() {
    echo "Build OS: ${@os.uname()[0].lower()}"
}
```

Real life example of anonymous function:
https://github.com/linux4sam/meta-atmel/blob/scarthgap/recipes-kernel/linux/linux.inc



Variable flags



Variable flags

- *Variable flags*, or *varflags*, are used to store extra information on tasks and variables.
- They are used to control task functionalities.
- A typical example:

```
SRC_URI[sha256sum] = "97b2c3fb..."
```

- See the list of varflags supported by bitbake.
- More varflags can be added freely.



Variable flags examples

- `dirs`: directories that should be created before the task runs. The last one becomes the work directory for the task. Example: `do_fetch` in `base.bbclass`.

```
do_compile[dirs] = "${B}"
```

- `noexec`: disable the execution of the task.

```
do_settime[noexec] = "1"
```

- `nostamp`: do not create a *stamp* file when running the task. The task will always be executed.

```
do_menuconfig[nostamp] = "1"
```

- `doc`: task documentation displayed by *listtasks*.

```
do_settime[doc] = "Set the current time in ${TIME}"
```

- `depends`: add a dependency between specific tasks

```
do_patch[depends] = "quilt-native:do_populate_sysroot"
```



Package features



Benefits

- Features can be built depending on the needs.
- This allows to avoid compiling all features in a software component when only a few are required.
- A good example is `ConnMan`: Bluetooth support is built only if there is Bluetooth on the target.
- The PACKAGECONFIG variable is used to configure the build on a per feature granularity, for packages.



PACKAGECONFIG

- PACKAGECONFIG takes the list of features to enable.
- `PACKAGECONFIG[<feature>]` takes up to six arguments, separated by commas:
  1. Argument used by the configuration task if the feature is enabled (EXTRA_OECONF).
  2. Argument added to EXTRA_OECONF if the feature is disabled.
  3. Additional build dependency (DEPENDS), if enabled.
  4. Additional runtime dependency (RDEPENDS), if enabled.
  5. Additional runtime recommendations (RRECOMMENDS), if enabled.
  6. Any conflicting PACKAGECONFIG settings for this feature.
- Unused arguments can be omitted or left blank.



Example: from ConnMan

```
PACKAGECONFIG ??= "wifi openvpn"

PACKAGECONFIG[wifi] = "--enable-wifi,          \
                       --disable-wifi,         \
                       wpa-supplicant,         \
                       wpa-supplicant"
PACKAGECONFIG[bluez] = "--enable-bluetooth,    \
                        --disable-bluetooth,   \
                        bluez5,                \
                        bluez5"
PACKAGECONFIG[openvpn] = "--enable-openvpn,    \
                          --disable-openvpn,   \
                          ,                    \
                          openvpn"
```



Enabling PACKAGECONFIG features

- In a `.bbappend` of the recipe, just append to PACKAGECONFIG

```
PACKAGECONFIG += "<feature>"
PACKAGECONFIG += "tui"
```

- In a config file (e.g. distro conf)

```
PACKAGECONFIG:append:pn-<recipename> = " <feature>"
PACKAGECONFIG:append:pn-gdb = " tui"
```



Inspecting available PACKAGECONFIG flags

- ${OECORE_DIR}/scripts/contrib/list-packageconfig-flags.py shows the PACKAGECONFIG varflags available for each recipe:

```
$ ../openembedded-core/scripts/contrib/list-packageconfig-flags.py
RECIPE NAME    PACKAGECONFIG FLAGS
================================
alsa-plugins   aaf jack libav maemo-plugin maemo-resource-manager pulseaudio samplerate speexdsp
connman        3g bluez client iptables l2tp nfc nftables openvpn pptp systemd tist vpnc wifi ...
gdb            babeltrace debuginfod python readline tui xz
...
```

- The -a flag shows all the details:

```
$ ../openembedded-core/scripts/contrib/list-packageconfig-flags.py -a
connman-1.41
/home/murray/w/yocto-stm32-labs/openembedded-core/meta/recipes-connectivity/connman/connman_1.41.bb
PACKAGECONFIG wispr iptables client                  3g wifi                    bluez
PACKAGECONFIG[wifi] --enable-wifi, --disable-wifi, wpa-supplicant, wpa-supplicant
PACKAGECONFIG[bluez] --enable-bluetooth, --disable-bluetooth, bluez5, bluez5
PACKAGECONFIG[openvpn] --enable-openvpn --with-openvpn=${sbindir}/openvpn,--disable-openvpn,,openvpn
...
```



Conditional features



Conditional features

- Some values can be set dynamically, thanks to a set of functions:
- `bb.utils.contains(variable, checkval, trueval, falseval, d)`: if `checkval` is found in `variable`, `trueval` is returned; otherwise `falseval` is used. `d` is the BitBake datastore.
- `bb.utils.filter(variable, checkvalues, d)`: returns all the words in the variable that are present in the checkvalues.
- Example (`meta/recipes-connectivity/connman/connman.inc`):

```
PACKAGECONFIG ??= "wispr iptables client \
                   ${@bb.utils.filter('DISTRO_FEATURES', '3g systemd', d)} \
                   ${@bb.utils.contains('DISTRO_FEATURES', 'bluetooth', 'bluez', '', d)} \
                   ${@bb.utils.contains('DISTRO_FEATURES', 'wifi', 'wifi ${WIRELESS_DAEMON}', '', d)} \
"
```



Configuration fragments



Configuration fragments

- Allow to group consistent configuration pieces together
- The idea is to ease the configuration, avoiding users to manually modify configuration files
- The `bitbake-config-build` utility can be used to enable or disable a configuration fragment in `toolcfg.conf`
- Each layer can provide its own configuration fragments
- Example: Adding the `root-login-with-empty-password`:
  - Provided by openembedded-core
  - Allows to log as root but also to have a root user with an empty password

```
bitbake-config-build enable-fragment core/yocto/root-login-with-empty-password
```



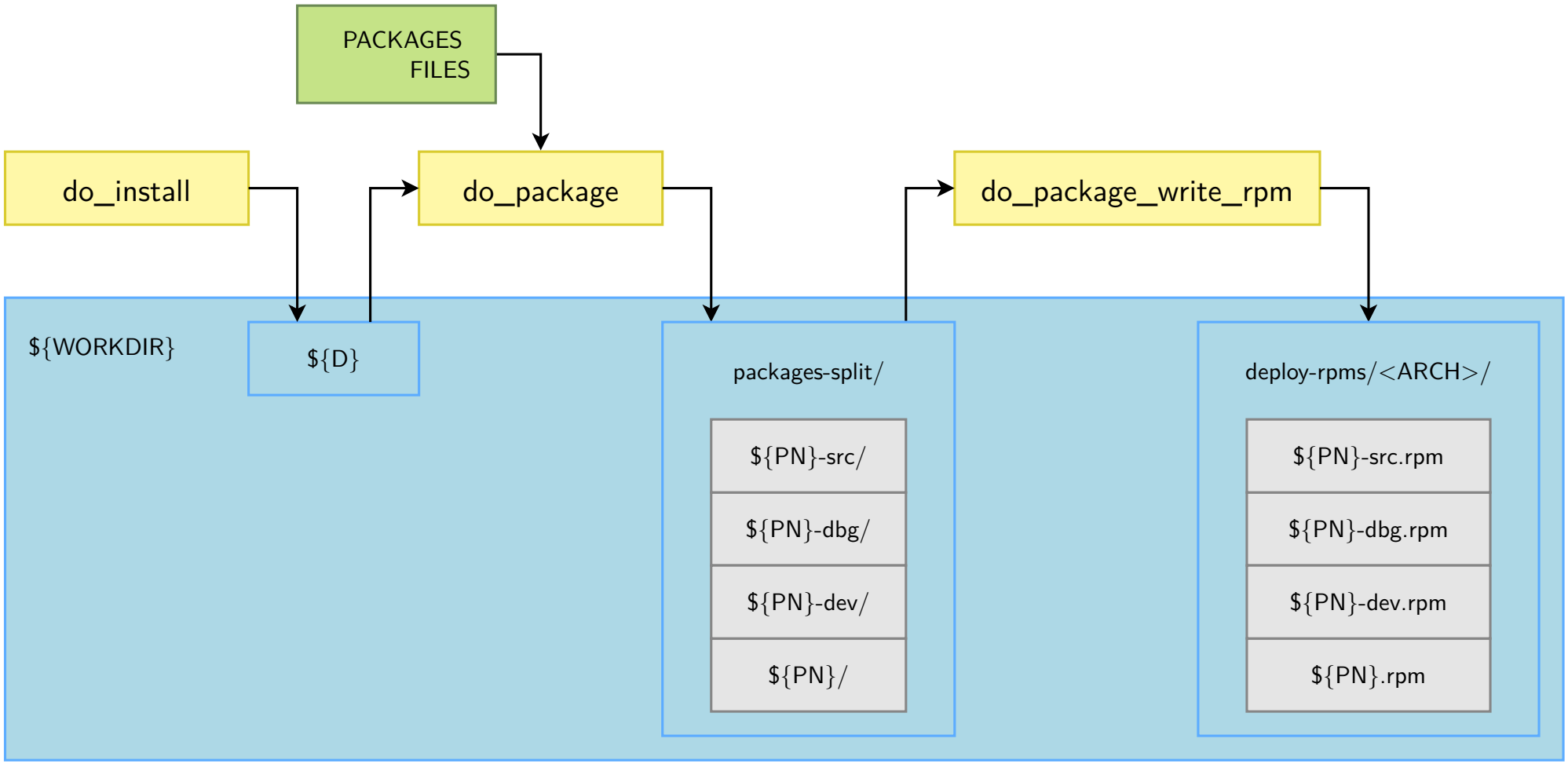
Built-in configuration fragments

- Bitbake support fragments that directly map to a variable
  - E.g. enabling fragment `foo/bar` might be equivalent to using `FOO = "bar"`
- The list of built-in configuration fragments is controlled by `OE_FRAGMENTS_BUILTIN`
- Openembedded-core implements this for `MACHINE` and `DISTRO` variables
  - Using fragments `distro/poky` and `machine/qemux86-64` is equivalent to adding `DISTRO = "poky"` and `MACHINE = "qemux86-64"` in your `local.conf`
- This allows to set basic configurations without having to ever modify a configuration file manually



Package splitting

Package splitting





Package splitting

- `do_install` copies *all* files into the `D` directory (`${WORKDIR}/image`).
- `do_package` splits files in several packages in `${WORKDIR}/packages-split`
  - based on the PACKAGES and FILES variables.
- `do_package_write_rpm` generates RPM packages



PACKAGES

- PACKAGES lists the packages to be built:

```
PACKAGES = "${PN}-src ${PN}-dbg ${PN}-staticdev ${PN}-dev \
    ${PN}-doc ${PN}-locale ${PACKAGE_BEFORE_PN} ${PN}"
```

- More packages can be added to the default list
  - Useful when a single remote repository provides multiple binaries or libraries.
  - The order matters. PACKAGE_BEFORE_PN allows to pick files normally included in the default package in another.
- PACKAGES_DYNAMIC allows to check dependencies when optional packages are satisfied.
- ALLOW_EMPTY allows to produce a package even if it is empty.
- To prevent configuration files from being overwritten during the Package Management System update process, use CONFFILES.



FILES

- For each package a FILES variable lists the files to include.
- It must be package specific (e.g. with `:${PN}`, `:${PN}-dev`, ...).
- Defaults from `meta/conf/bitbake.conf`:

```
FILES:${PN}-dev = "\
     ${includedir} ${FILES_SOLIBSDEV} ${libdir}/*.la \
     ${libdir}/*.o ${libdir}/pkgconfig ${datadir}/pkgconfig \
     ${datadir}/aclocal ${base_libdir}/*.o \
     ${libdir}/${BPN}/*.la ${base_libdir}/*.la \
     ${libdir}/cmake ${datadir}/cmake"
FILES:${PN}-dbg = "\
     /usr/lib/debug /usr/lib/debug-static \
     /usr/src/debug"
```



FILES: the main package

- The package named just `${PN}` is the one that gets installed in the root filesystem.
- In Openembedded-core, defaults to:

```
FILES:${PN} = "\
     ${bindir}/* ${sbindir}/* ${libexecdir}/* ${libdir}/lib*${SOLIBS} \
     ${sysconfdir} ${sharedstatedir} ${localstatedir} \
     ${base_bindir}/* ${base_sbindir}/* \
     ${base_libdir}/*${SOLIBS} \
     ${base_prefix}/lib/udev/rules.d ${prefix}/lib/udev/rules.d \
     ${datadir}/${BPN} ${libdir}/${BPN}/* \
     ${datadir}/pixmaps ${datadir}/applications \
     ${datadir}/idl ${datadir}/omf ${datadir}/sounds \
     ${libdir}/bonobo/servers"
```



Example

- The `kexec-tools` **recipe** provides the `kexec` and `kdump` **packages**:

```
require kexec-tools.inc
export LDFLAGS = "-L${STAGING_LIBDIR}"
EXTRA_OECONF = " --with-zlib=yes"

SRC_URI[sha256sum] = "467ba3fa52ef..."

PACKAGES =+ "kexec kdump"

FILES:kexec = "${sbindir}/kexec"
FILES:kdump = "${sbindir}/kdump"
```



Inspecting packages

`oe-pkgdata-util` is a tool that can help inspecting packages:

- Which package is shipping a file:

```
$ oe-pkgdata-util find-path /bin/busybox
busybox: /bin/busybox
```

- Which files are shipped by a package:

```
$ oe-pkgdata-util list-pkg-files busybox
busybox:
    /bin/busybox
    /bin/busybox.nosuid
    /bin/busybox.suid
    /bin/sh
```

- Which recipe is creating a package:

```
$ oe-pkgdata-util lookup-recipe kdump
kexec-tools
$ oe-pkgdata-util lookup-recipe libtinfo5
ncurses
```



Dependencies in detail



DEPENDS

- DEPENDS describes a build-time dependency
- Typical case: a program needs the library and headers files from a library to be configured and/or built
- In other words: it needs the library in its *sysroot*
- In `ninvaders.bb`, the line
  `DEPENDS = "ncurses"`
  creates a dependency
  - Of `ninvaders.do_prepare_recipe_sysroot`
  - On `ncurses.do_populate_sysroot`



RDEPENDS

- RDEPENDS describes a runtime dependency
- Typical case: a program uses another program at runtime via sockets, DBUS, etc, or simply executes it
- It does not need it at build time
- In `inetutils_2.4.bb`, the line
  `RDEPENDS:${PN}-ftpd += "xinetd"`
  creates a dependency
  - Of `inetutils.do_build`
  - On `xinetd.do_package_write_rpm`
- And adds in the `inetutils-ftpd` RPM package a dependency on the `xinetd` RPM package



RRECOMMENDS

- RRECOMMENDS is similar to RDEPENDS
- But if the dependency package is not built it will just be skipped instead of failing the build
- Typical cases:
  - A package extends the features of a program, but its build has been disabled explicitly (e.g. via BAD_RECOMMENDATIONS)
  - Depending on a kernel module that might also be built-in in the kernel Image
- In `watchdog_5.16.bb`, the line
  `RRECOMMENDS:${PN} += "kernel-module-softdog"`
  does nothing if the `softdog` kernel module is not built by the kernel (could be builtin)



Inspecting dependencies

- Dependencies can be inspected by generating a dot file:

```
$ bitbake -g core-image-minimal
...
NOTE: Task dependencies saved to 'task-depends.dot'
```

- The file can be read manually, or `oe-depends-dot` can be used:

```
$ oe-depends-dot --key busybox --why ./task-depends.dot
Because: core-image-minimal packagegroup-core-boot
core-image-minimal -> packagegroup-core-boot -> busybox
$ oe-depends-dot --key busybox --depends ./task-depends.dot
Depends: patch-native glibc libxcrypt <...>
```

- Or a graphical tool can be used to explore dependencies or reverse dependencies:
  - `bitbake -g -u taskexp_ncurses core-image-minimal`